



PROGRAMACIÓN I

Pseudocódigo

María Janneth Rivera Reyna

Licenciatura en Ciencias de la Computación

Departamento de Matemáticas

Universidad de Sonora

Ciclo 2026-2

Pseudocódigo

Es una **técnica** que nos permite **describir los pasos** para la **resolución de un problema**.

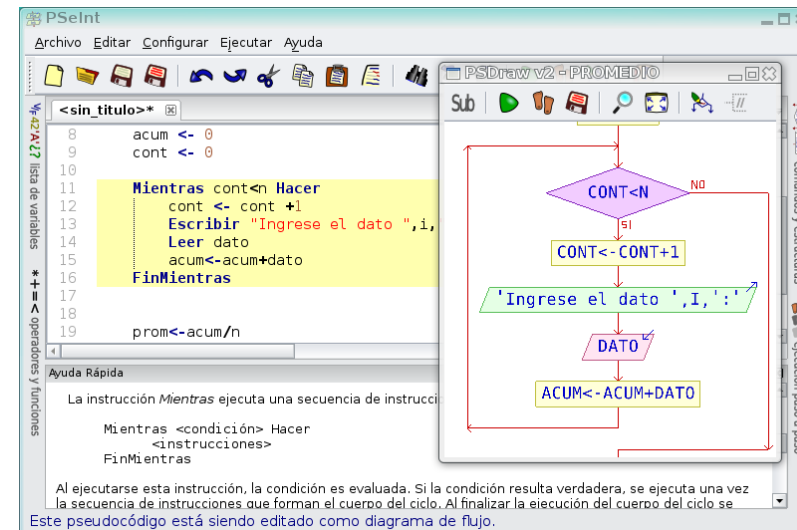
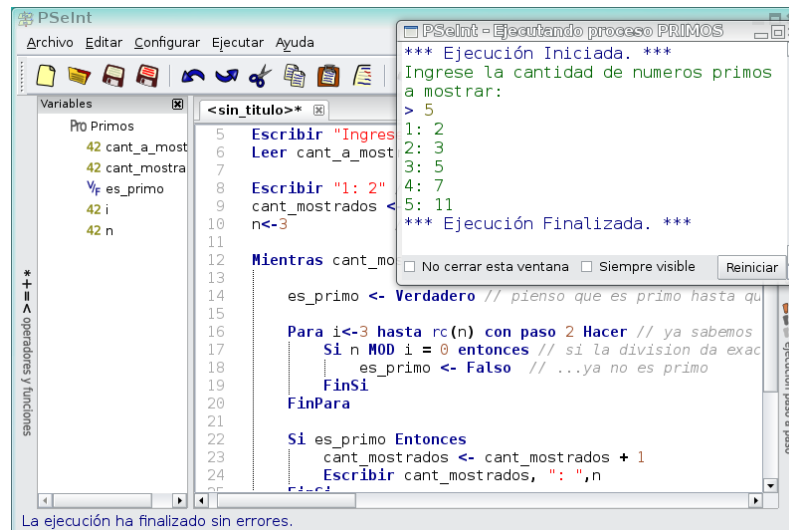
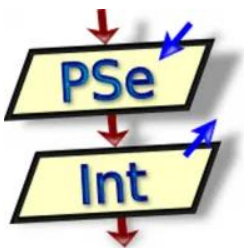
No se considera un lenguaje de programación, sino más bien **es un lenguaje intermedio** entre el lenguaje natural humano y el lenguaje de computadora:

- Es lo suficientemente **parecido a al lenguaje natural** (en nuestro caso, al español)
- Requiere del **uso de ciertas reglas** para expresar las ideas de una forma lógica, pero sin preocuparnos por la sintaxis de un lenguaje de programación.

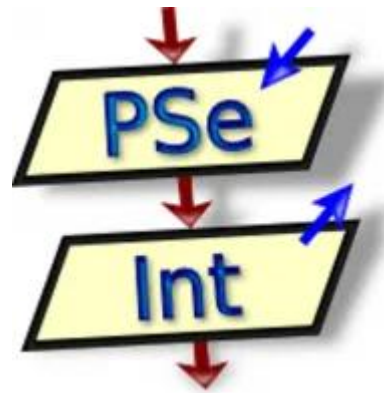
No se puede ejecutar en una computadora, sin embargo **existen programas didácticos** que nos ayudan a **escribir, probar y ejecutar algoritmos** y ver que funcionen de forma correcta.

PSeInt

- Software libre educativo multiplataforma dirigido a personas que inician en programación.
- Creado por Pablo Novara en 2003, como proyecto final de la materia de Programación I, en la Universidad Nacional del Litoral, en Argentina.
- Es utilizado principalmente por estudiantes para aprender los **fundamentos de la programación y el desarrollo de la lógica, usando pseudocódigo y diagramas de flujo.**



Elementos básicos en PSeInt



VARIABLE

Es un **espacio en la memoria RAM** de la computadora que sirve para **almacenar un dato o valor**.

Tiene 3 atributos:

- **Nombre o Identificador:** Permite identificar un dato para distinguirlo de otros y poder manipularlo.
- **Contenido:** Es el valor que contiene el dato.
- **Tipo:** El tipo de dato asociado.

IDENTIFICADORES

Nombres usados para hacer **referencia a las variables**.

REGLAS

- Puede contener sólo:
 - Mayúsculas y minúsculas [a-z] [A-Z], excepto ñÑ
 - Dígitos [0-9]
 - Caracter de subrayado (_)
- Puede iniciar con letra o (_), no con dígito
- No debe ser una palabra reservada de PSeInt
- No hace distinción entre mayúsculas y minúsculas
- No debe contener espacios

Palabras reservadas en PSeInt

Algoritmo	FinAlgoritmo	Proceso	FinProceso
Definir	Como	Entero	Enteros
Caracter	Caracteres	Cadena	Cadenas
Leer	Si	Entonces	SiNo
FinSi	Segun	Hacer	De Otro Modo
FinSegun	Mientras	FinMientras	Repetir
Hasta Que	Para	Con Paso	FinPara
Funcion	FinFuncion	Fin	Verdadero
Falso	Y	O	No
Mod	Div	Limpiar Pantalla	Esperar Tecla
Dimension	Mostrar	Imprimir	es
Son	Real	Reales	Texto
Logico	Logicos	Numero	Numerico
Logica	Logicas	Entera	Enteras
Escribir	Sin	Saltar	Limpiar
Pantalla	opcion	valor	

IDENTIFICADORES

Identificador	Válido	Detalle
suma enteros		El espacio en blanco
Año		La ñ
calificación		La letra ó con acento
2c		Iniciar con dígito
precio-total		El guión
nombreUsuari@		La @
Nombre.apellido		El punto
a+b		El +
_contador		Es válido
RESULTADO_1		Es válido

TIPOS DE DATOS SIMPLES

PSelnt soporta 3 tipos de datos simples:

- **Numérico:** Almacena valores numéricos enteros o reales (usan el punto para separar decimales). Para definir este tipo de dato se usa **Número**, **Numérico**, **Real**, **Entero**.
- **Lógico:** Puede contener sólo los valores **FALSO** y **VERDADERO**, aunque también puede usarse **F** y **V**, ó **0** y **1**.
- **Cadena:** Pueden contener un caracter o una cadena de caracteres encerrados entre comillas (simples o dobles). Se define como **Caracter**, **texto**, **cadena**.

TIPOS DE DATOS SIMPLES

Tipo de dato	Ejemplos
Entero	<i>1, 4, -6, 342, -32021, 0</i>
Real	<i>-238.423, 2.0, 3.1542, 0.00001</i>
Caracter	<i>"{", "*", "9", "Universidad de Sonora", "Mi teléfono es +52 (662) 123-45-67", 'Hola', 'S'</i>

DECLARACIÓN DE VARIABLES

- Para poder utilizar una variable en un algoritmo, es necesario **declararla**.
- Cuando declaramos una variable, le estamos anunciando a la computadora **qué tipo de datos** vamos a almacenar en dicha variable.

FORMATO

DEFINIR <var> **COMO** [tipo singular]

DEFINIR <var1, var2, ...> **COMO** [tipo plural]

Ejemplos:

Definir edad **Como Entero**

Definir edad1, edad2 **Como Enteros**

BUENAS PRÁCTICAS

IDENTIFICADORES

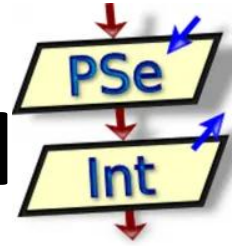
- Usar identificadores que estén relacionados con lo que representan en el algoritmo para que sea mas legible.

Ejemplo:

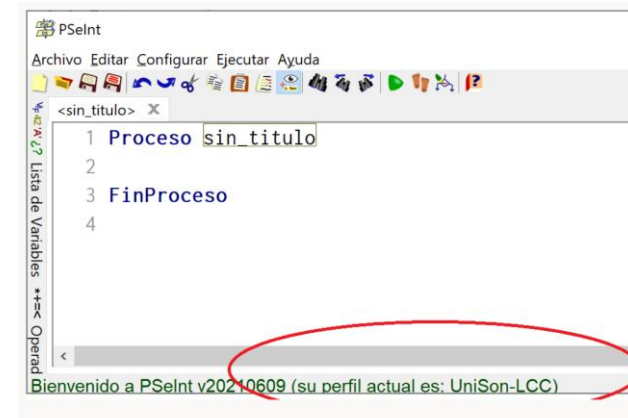
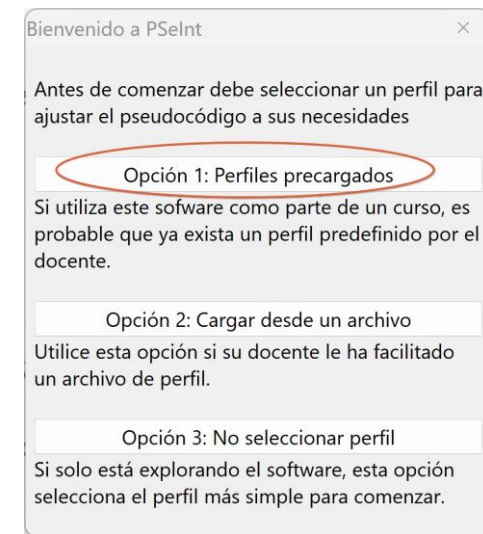


- Si se va a declarar una variable que guarde una calificación, podría usarse el identificador “calif” o bien, “calificacion”.
Usar “c”, “c1”, “clf” podría no ser muy intuitivo.

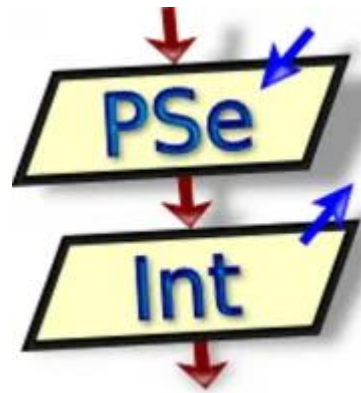
PSeInt: Instalación y configuración del perfil



1. Visita la página de [PSeInt](#) y descarga para tu sistema operativo.
2. Si es la primera vez que lo instalas en tu computadora, verás el siguiente menú, elige **Opción 1: Perfiles precargados**.
3. Si ya lo tienes instalado, ve al menú **Configurar > Opciones del lenguaje (perfiles)**
4. En **Buscar**, escribe: **UniSon-LCC** y selecciona. Haz click en **Aceptar**.
5. Asegúrate que en la parte inferior el perfil actual sea Unison-LCC.
6. **Como evidencia**, sube en tu tarea asignada en Teams, una captura de tu pantalla como la siguiente:



Ejercicios: Declaración de variables



¡Manos a la obra!

ESCRITURA DE DATOS

Esta operación permite a la máquina presentarle datos al usuario.

FORMATO

ESCRIBIR <var>

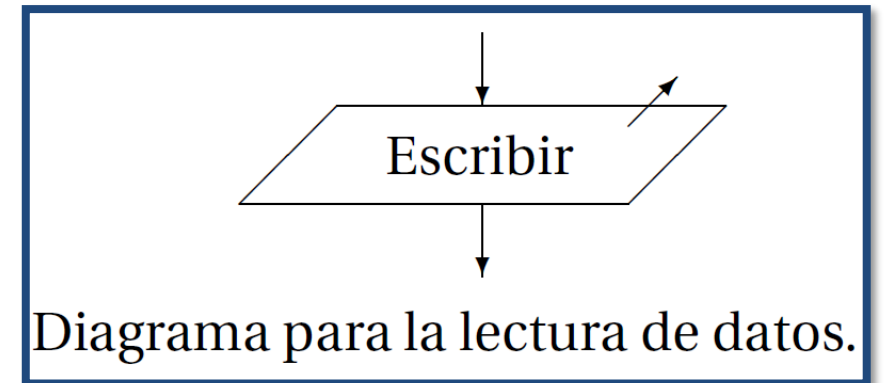
Ejemplo:

ESCRIBIR dato1

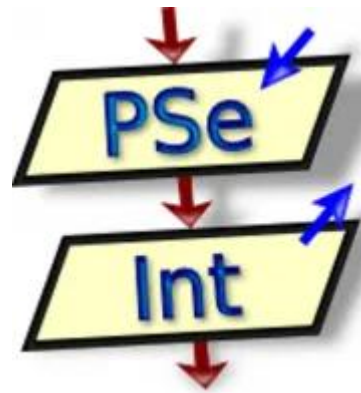
Donde:

ESCRIBIR → especifica la operación a realizar

'dato1' → es el nombre de la variable que contiene
la información que se quiere presentar al usuario



Ejercicios: Escritura de variables



¡Manos a la obra!

LECTURA DE VARIABLES

Con esta operación se captura o se recibe la información dada por el usuario.

FORMATO

LEER <var>

Ejemplo:

LEER dato1

Donde:

LEER → especifica la operación a realizar

'dato1' → es el nombre de la variable donde se almacenará la información leída.

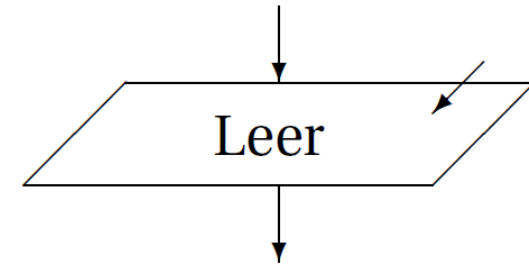
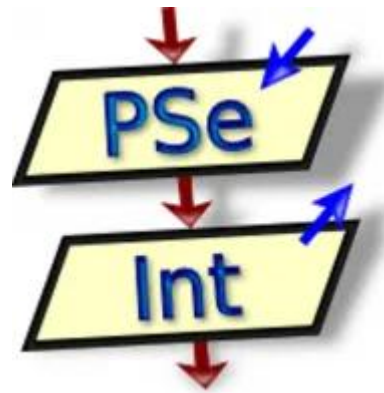


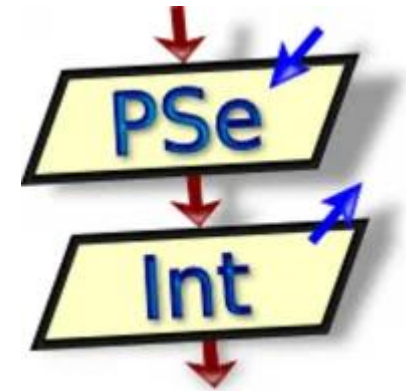
Diagrama para la lectura de datos.

Ejercicios: Lectura de variables



¡Manos a la obra!

Práctica 1



- En PSeInt, escribe el pseudocódigo para preguntarle al usuario la siguiente información:
 - Nombre
 - Edad
 - Carrera que cursa
 - Hobbies / Pasatiempo favorito
 - Usuario de correo (por ejemplo: “janneth”)
 - Dominio del servidor de correo (por ejemplo: “@unison.mx”)
 - Mostrar texto: “Tu correo es: janneth@unison.mx”

OPERADORES ARITMÉTICOS

Operación	Operador
Suma	+
Resta	—
Multiplicación	*
División (real)	/
Módulo	MOD
Asignación	:= <-

- Todos los operadores (excepto el de asignación) se aplican de modo *binario* (con dos operadores) y utilizan operandos constantes o variables a la izquierda y a la derecha del operador:

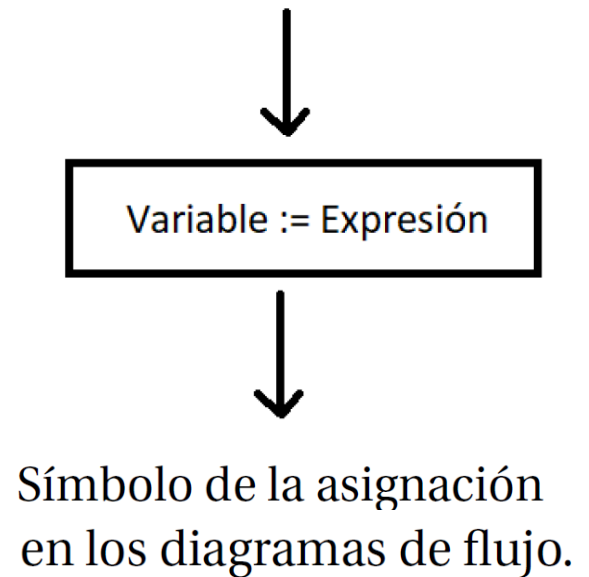
- 5+4
- b-a
- a*2
- 20/5
- 20MOD4

OPERADORES ARITMÉTICOS

- El operador de **asignación** sirve para asignar el valor de la derecha a la variable de la izquierda:

variable := valor asignado

x := 2	
x := y	
x := 2+y	
x := aleatorio(1,5)	
letra := 'a'	
cad := 'Juan'	



OPERADORES ARITMÉTICOS

Los operadores $+$ y $-$ también pueden ser aplicados en forma *monaria*, es decir, con un operando.

Por ejemplo:

$x1 := -2$	asigna el valor constante -2 a x1
$x2 := +3$	asigna el valor constante 3 a x2

OPERADORES ARITMÉTICOS

Las operaciones deben escribirse en una sola línea.

Por ejemplo, si se tiene la fórmula:

$$x = \frac{1}{a} + \frac{1}{b}$$

debe transformarse para que tome la forma:

$$x = 1/a + 1/b$$

OPERADORES ARITMÉTICOS

Hay ocasiones en que la transformación no es tan directa.

Por ejemplo, la ecuación:

$$x = \frac{a + b}{2} c$$

puede transformarse a:

$$\begin{aligned} x &= (a + b) * c / 2 \\ x &= (a + b) / 2 * c \end{aligned}$$

OPERADORES ARITMÉTICOS

Para hacer la transformación correcta, es necesario conocer la **precedencia de los operadores**, es decir, saber qué operador se debe ejecutar primero:

Nivel de precedencia	Operadores	Asociatividad
1	()	
2	+ - (monarios)	Derecha a izquierda
3	* / MOD	Izquierda a derecha
4	+ - (binarios)	Izquierda a derecha
5	:=	Derecha a izquierda

Los que están hasta arriba de la tabla tienen mayor precedencia. Si se quiere modificar la precedencia, se utilizan los paréntesis.

Por ejemplo:

$$x = \frac{2 + 5a}{-\frac{a}{2} + ab}$$

Se transformaría a:

$$x = (2 + 5 * a) / (-a / 2 + a * b)$$

Ejercicio: Calcular el área de un triángulo

$$A = \frac{ba}{2} \longrightarrow a := base * altura / 2$$

1. **Identifiquen:**
 - a. Entrada
 - b. Proceso
 - c. Salida
2. **Dibujen** el diagrama de flujo.
3. **Escriban** el pseudocódigo de este algoritmo.



COMENTARIOS

Los comentarios son **notas explicativas** pensadas para ser leídas por un futuro lector (que puede ser alguien más o el mismo programador).

El objetivo es hacer **más legible el código**, sin afectar su funcionamiento, ya que este texto no se ejecuta.

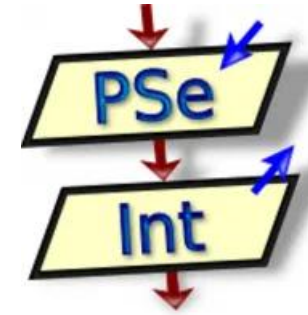
FORMATO

```
//Este es mi primer comentario
```

Donde:

Todo lo que quede a la derecha del símbolo `//` se considera comentario hasta el final de esa línea.

Práctica 2



En PSeInt:

1. **Escribe** el pseudocódigo para calcular el área de un triángulo. Utiliza como base **el diagrama de flujo y el pseudocódigo** realizado anteriormente en clase.
2. **Escribe comentarios en la cabecera del programa:**
 - a) Nombre del archivo
 - b) Autor
 - c) Descripción
3. **Comenta cada línea** para indicar lo que se está haciendo.
4. **Crea y guarda el diagrama de flujo** generado por PSeInt. ¿Se parece al que hiciste en clase?

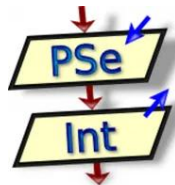
Práctica 3a: Calcular el área de un círculo

$$A = \pi * r^2 \longrightarrow A = pi * radio * radio$$



En su cuaderno:

1. **Identificar:**
 - a. Entrada
 - b. Proceso
 - c. Salida
2. **Dibujar** el diagrama de flujo.
3. **Escribir** el pseudocódigo de este algoritmo.

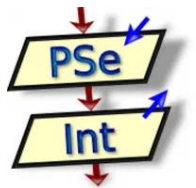


En PSeInt:

1. **Escribir** el pseudocódigo de este algoritmo.
Leer un valor aproximado para pi (p.ej. 3.14)
2. **Agregar comentarios en cabecera.**
 - a) Nombre del archivo
 - b) Autor
 - c) Descripción
3. **Comentar cada línea** describiendo lo que se está haciendo.
4. **Generar y guardar diagrama de flujo.**

Práctica 3b: Calcular el área de un círculo

$$A = \pi * r^2 \longrightarrow A = PI * radio \uparrow 2$$



En PSeInt:

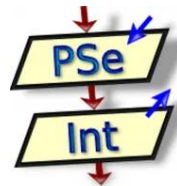
1. Repetir pseudocódigo anterior, pero ahora utilizando:
 - a) La constante **PI** definida en PSeInt
 - b) El operador de **potencia** $\uparrow$

Práctica 4: Calcular el promedio de 3 calificaciones



En su cuaderno:

1. **Identificar:**
 - a. Entrada
 - b. Proceso
 - c. Salida
2. **Dibujar** el diagrama de flujo.
3. **Escribir** el pseudocódigo de este algoritmo.



En PSeInt:

1. **Escribir** el pseudocódigo de este algoritmo.
2. **Agregar comentarios en cabecera.**
 - a) Nombre del archivo
 - b) Autor
 - c) Descripción
3. **Comentar cada línea** describiendo lo que se está haciendo.
4. **Generar y guardar diagrama de flujo.**

OPERADORES ARITMÉTICOS: MOD

- El operador de modulo MOD, calcula el **residuo de una division entera**.

$$\begin{array}{r} 10 \\ 2 \overline{) 21} \\ \underline{20} \\ 1 \end{array}$$

Residuo

- Ejemplos:

- $21 \text{ MOD } 2 = 1$, porque 2 cabe 10 veces en 21 y sobra 1.
- $10 \text{ MOD } 3 = 1$, porque 3 cabe 3 veces en 10 y sobra 1.
- $7 \text{ MOD } 4 = 3$, porque 4 cabe 1 vez en 7 y sobra 3.
- $10 \text{ MOD } 2 = 0$, porque 10 entre 2 es exacto y no sobra nada.
- $5 \text{ MOD } 10 = 5$, porque el 10 no cabe ninguna vez en el 5 y el total del dividendo pasa a ser el residuo.

OPERADORES ARITMÉTICOS: MOD

Ejemplo 1

Proceso OperacionesEntreEnteros

Definir x1,x2 **Como** Entero

Escribir "Proporciona un valor para x1"

Leer x1

Escribir "Proporciona un valor para x2"

Leer x2

Escribir "El cociente de la división entera entre ",x1," y ",x2," es ",trunc(x1/x2)

Escribir "El residuo de la división entera entre ",x1," y ",x2," es ",x1 mod x2

FinProceso

OPERADORES RELACIONALES

Sirven para formar expresiones lógicas cuyo valor es **FALSO** o **VERDADERO**.

A estos valores también se les conoce como **valores de verdad** o **de certeza**.

Símbolo matemático	Nombre	Símbolo en pseudocódigo
$<$	Menor que	$<$
$\leq$	Menor o igual que	$\leq$
$>$	Mayor que	$>$
$\geq$	Mayor o igual que	$\geq$
$=$	Igualdad lógico	$=, ==$
$\neq$	Desigualdad lógica	$<>, !=$

OPERADORES RELACIONALES

Ejemplo 1:

Considere la siguiente expresión: $a < b$

Cuyo valor puede ser FALSO o VERDADERO, dependiendo del valor de a y b

a	b	$a < b$
4	10	VERDADERO
5	1	FALSO
5	5	FALSO

OPERADORES RELACIONALES

Ejemplo 2:

Considere la siguiente expresión: $a \geq 'f'$

Donde se compara el contenido de una variable con respecto a un caracter.

La variable a puede ser de tipo **entero** o **carácter**, en ambos se utiliza para la comparación con el orden establecido en la tabla de códigos ASCII.

El código ASCII de 'f' es 102.

a	a >= 'f'
'c'	FALSO
'r'	VERDADERO
20	FALSO

OPERADORES LÓGICOS

A las expresiones anteriores se les conoce como **expresión lógica simple**. Usando expresiones lógicas simples, podemos obtener **expresiones lógicas compuestas**.

Una expresión lógica compuesta se forma utilizando operadores relacionales y los operadores lógicos **AND, OR y NOT**.

Los operadores AND y OR requieren dos operandos lógicos, mientras que el operador NOT requiere un solo operando; dichos operandos generalmente surgen como resultado de evaluar expresiones lógicas simples, y dan como resultado otro valor de verdad.

Nombre	Símbolo pseudocódigo
AND	Y, &, &&
OR	O, ,
NOT	NO, !

OPERADORES LÓGICOS: AND

Tabla de verdad del operador lógico AND.

a	b	a Y b
V	V	V
V	F	F
F	V	F
F	F	F

Por ejemplo, considere la expresión compuesta:

$(a > b) \text{ Y } (c \leq d)$

Si $a=10$, $b=3$, $c=1$, y $d=1$

$\begin{array}{cc} \text{v} & \text{v} \\ \text{---} & \text{---} \\ & \text{v} \end{array}$

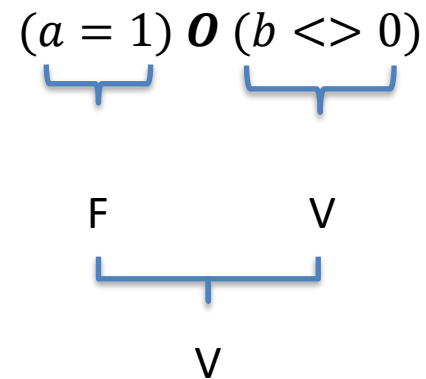
OPERADORES LÓGICOS: OR

Tabla de verdad del operador lógico OR.

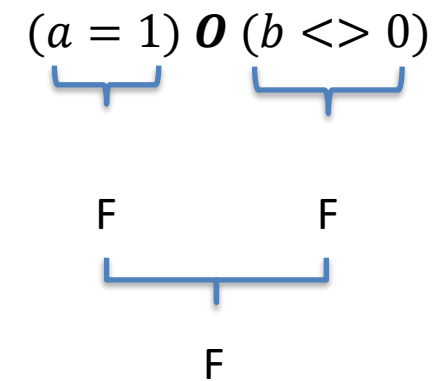
a	b	a O b
V	V	V
V	F	V
F	V	V
F	F	F

Por ejemplo, considere la expresión compuesta:

Si $a=7$, $b=15$,



¿Y si $a=6$, $b=0$?



OPERADORES LÓGICOS: NOT

Tabla de verdad del operador lógico NO.

x	NO x
V	F
F	V

Por ejemplo, la expresión:

NO ($a < 6$)

Si $a=10$  V

Si $a=1$  F

OPERADORES LÓGICOS: LEYES de DE MORGAN

Leyes de DeMorgan para la lógica.

$\text{NO}(x1 \text{ Y } x2) \equiv \text{NO } x1 \text{ O NO } x2$
$\text{NO}(x1 \text{ O } x2) \equiv \text{NO}x1 \text{ Y NO } x2$

Hacen posible escribir una expresión lógica compuesta de **dos formas diferentes** y, sin embargo, ser **equivalentes**.

Por ejemplo:

$$\begin{aligned} & \text{NO}(x+y>0 \text{ Y } x\geq 0) \\ & \equiv \text{NO}(x+y>0) \text{ O NO}(x\geq 0) \\ & \equiv (x+y\leq 0) \text{ Y } (x<0) \end{aligned}$$

PRECEDENCIA DE OPERADORES (actualización)

NO(x+y>0 Y x>=0)

En la expresión anterior, usamos paréntesis en cada expresión donde se aplica una operación **NOT** para indicar que ésta, será evaluada después de la expresión que aparece dentro de ellos.

Pero no utilizamos paréntesis para indicar que la suma debe realizarse antes de la comparación en la expresión x+y>0. Esto se debe a que el operador + tiene mayor precedencia que >.

Precedencia de los operadores.

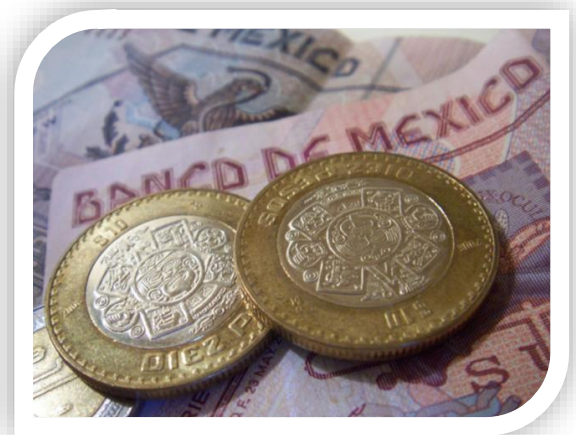
Operadores			
()			
NOT	+	-	(monarios)
* / MOD			
+ - (binarios)			
< <= > >=			
<> =			
Y, &, &&			
O, ,			

Práctica 5a: Separar dinero

PROBLEMA: Separar una cantidad en billetes de \$100, monedas de \$10 y de \$1 peso.

INSTRUCCIONES:

1. Leer la cantidad que se desea descomponer:
`cantidad`
2. Para separar en billetes de 100, hay que dividir la cantidad entre 100 con división entera:
`billetes100 := trunc (cantidad/100)`
3. Una vez obtenido el número de billetes de 100, hay que calcular el resto del dinero:
`resto = cantidad mod 100`
4. Con el resto se calcula el número de monedas de 10 que se requieren usando división entera:
`monedas10 := trunc (resto/10)`
5. Finalmente, se obtiene el número de monedas de 1 peso requeridas:
`monedas1 = resto mod 10`



Práctica 5b: Separar dinero, más denominaciones

PROBLEMA: Resuelve el ejercicio anterior, pero ahora incluye también las siguientes denominaciones:

- Billetes de 50
- Billetes de 20
- Monedas de 5

- No olvidar incluir cabecera:
 - Nombre del archivo:
 - Autor:
 - Descripción
- Incluir comentarios por línea
- Generar diagrama de flujo



Práctica 6: Operadores relacionales y lógicos

INSTRUCCIONES:

1. Pedir al usuario valores para: a y b (una sola vez)
2. Crear las siguientes expresiones lógicas compuestas:
 - a) $(a > b) \text{ Y } (b > a)$
 - b) $(a > b) \text{ O } (b > a)$
 - c) $\text{NO}(a > b)$
 - d) $(a \neq b) \text{ Y } (b > 5)$
 - e) $(a \neq b) \text{ O } (b > 5)$
 - f) $\text{NO}(a \neq b)$
3. Imprimir en pantalla cada expresión, sustituyendo a y b por los valores dados por el usuario y mostrando el valor de verdad resultante para cada expresión.

Ejemplo a):

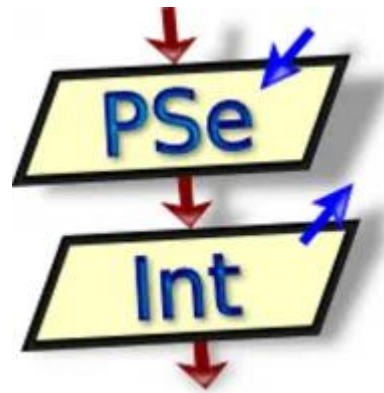
```
a = 5
b = 10
(5 > 10) Y (10 > 5) es Falso
```

Práctica 7: Operadores relacionales y lógicos

INSTRUCCIONES:

1. Pedir al usuario valores para 4 variables enteras a , b , c y d .
2. Crear las siguientes expresiones lógicas:
 - a) Los valores de a y b son ambos menores que 17.
 - b) Los valores de a , b y c son idénticos y distintos de d .
 - c) El valor de b es mayor que el valor de a y menor que c .
 - d) Entre los valores de a , b , y c hay al menos dos idénticos.
3. Imprimir en pantalla cada expresión, sustituyendo a , b , c y d por los valores dados por el usuario y mostrando el valor de verdad resultante para cada expresión. (Como en el ejercicio anterior).

Estructuras de Control en PSeInt



ESTRUCTURAS DE CONTROL

Hasta el momento, hemos visto ejercicios donde se inicia con la ejecución de la primera instrucción, seguido de la ejecución de la segunda instrucción, etc. Y además, cada una de las instrucciones se ejecuta una sola vez.

A estos algoritmos se les conoce como **algoritmos secuenciales**.

Esta forma de programar es útil cuando se trata de problemas relativamente sencillos, pero generalmente, es necesario utilizar otros mecanismos que nos permitan decidir entre la ejecución de una instrucción (o un conjunto de instrucciones) y otra, así como para permitir que se ejecuten en más de una ocasión.

ESTRUCTURAS DE CONTROL

Esto es precisamente lo que harán las estructuras de control:

Elegir qué instrucciones se van a ejecutar cada vez.

Existen tres tipos de estructuras de control:

Secuenciación
(secuenciales)

Selección
(condicionales)

Repetición
(iterativas)

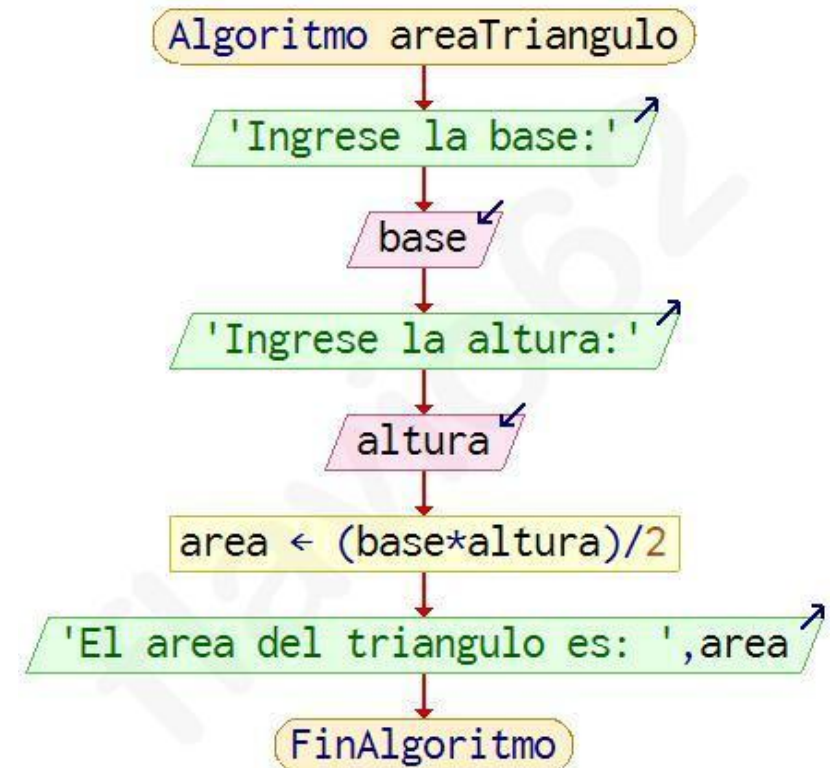


Estructuras de control

SECUENCIACIÓN

SECUENCIACIÓN (secuenciales)

- Las instrucciones se ejecutan **una tras otra**, en **orden lineal y continuo**.
- **No existen saltos ni decisiones**, por lo tanto, el flujo va del primer paso al último.





Estructuras de control

SELECCIÓN

SELECCIÓN (condicionales)

Permite seleccionar el **curso de ejecución** de algunas instrucciones del programa en base al cumplimiento de alguna **condición**.

Existen tres tipos de selección:

Simple

Doble

Múltiple

SELECCIÓN SIMPLE

Es una estructura de control que permite **decidir si se ejecuta o no**, un conjunto de instrucciones.

Dicha estructura se vale de una **expresión lógica (simple o compuesta)** llamada ***condición*** para decidir el flujo de acciones que se ejecutarán.

El formato es el siguiente:

Si *condición* **Entonces**
 ↳ Bloque de instrucciones a realizar cuando *condición* es verdadera
FinSi

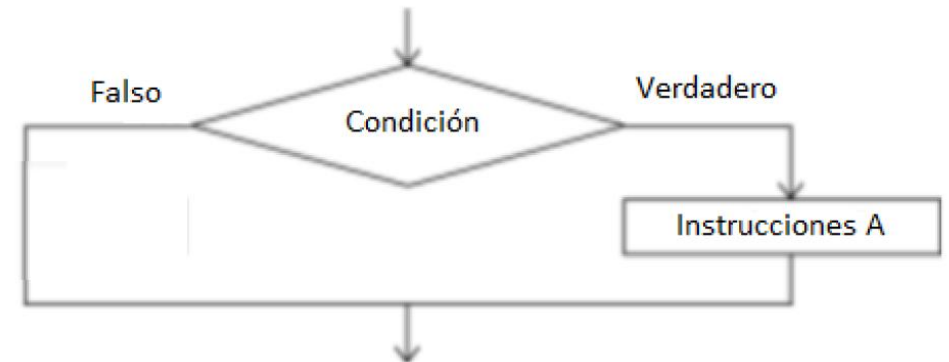
identación

SELECCIÓN SIMPLE

El flujo de acciones es el siguiente:

- Si *condición* es VERDADERO, entonces se ejecuta el bloque de instrucciones dentro de la estructura.
- En caso contrario se ignora.

El fin del bloque de instrucciones a ejecutar cuando *condición* es VERDADERO se especifica con la sentencia **FinSi**.



SELECCIÓN SIMPLE

Ejercicio: Felicitar si es mayor de edad

INSTRUCCIONES:

- Leer el nombre y la edad del usuario.
- Saludarle por su nombre.
- Si el usuario tiene 18 años, mandarle un mensaje de felicitación por se mayor de edad.

1. Identificar:

- Entrada: *Leer el nombre y la edad del usuario*
- Proceso: *Mostrar el nombre y comparar si edad es igual a 18*
- Salida: *Si edad es igual a 18 se manda un mensaje felicitando al usuario*

2. Dibujar diagrama de flujo



SELECCIÓN SIMPLE: Ejercicio

Diagrama de flujo



Práctica 8:

Felicitar si es mayor de edad



INSTRUCCIONES:

- Leer el nombre y la edad del usuario.
- Saludarlo por su nombre.
- Si el usuario tiene 18 años, mandarle un mensaje de felicitación.

En el cuaderno ya hicimos:



- ✓ Entrada
 - ✓ Proceso
 - ✓ Salida
-
- ✓ Diagrama de flujo

En PSeInt:

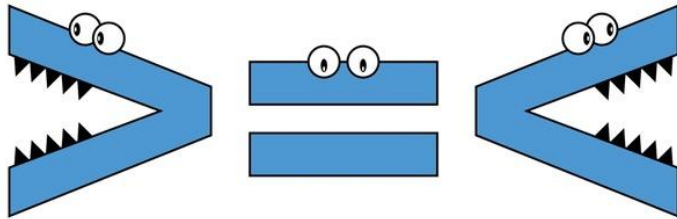


1. Escribir pseudocódigo.
 - a) Cabecera (Nombre archivo, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo
3. Guardar sus archivos.

Ejercicio: Menor de dos números (usando solo selección simple)

PROBLEMA:

Suponga que se tienen dos números enteros y se desea compararlos para imprimir en pantalla cuál es el menor de ellos.



En el cuaderno:



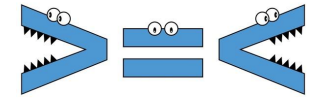
1. Identificar:

- Entrada: *Leer num1 y num2*
- Proceso: *Comparar si:*
 - a) $num1 < num2$
 - b) $num2 < num1$*¿Y si son iguales?*
 - c) $num1 = num2$
- Salida: *Mostrar mensaje indicando cuál número es menor, o si son iguales.*

2. Dibujar diagrama de flujo

Práctica 9:

Menor de dos números (usando sólo selección simple)



PROBLEMA:

Suponga que se tienen dos números enteros y se desea compararlos para imprimir en pantalla cuál es el menor de ellos.

En el cuaderno ya hicimos:



- ✓ Entrada
- ✓ Proceso
- ✓ Salida

- ✓ Diagrama de flujo

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 10:

Determinar si un número entero es par (usando sólo selección simple)

PROBLEMA:

Se le pide un número entero al usuario y se quiere indicar en pantalla si es par o impar.

En el cuaderno:



1. **Identificar:**
 - a) Entrada
 - b) Proceso
 - c) Salida
2. **Diagrama de flujo**

En PSeInt:



1. **Escribir pseudocódigo.**
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. **Generar diagrama de flujo.**
3. **Guardar sus archivos.**

SELECCIÓN DOBLE

La selección simple está limitada a especificar las acciones a realizar cuando el valor lógico de la condición es VERDADERO, pero no especifica qué hacer cuando el valor lógico de la condición es FALSO.

La selección doble **considera ambos casos**, pues especifica las operaciones a realizar para cada uno de los valores de la condición lógica.

El formato es el siguiente:

Si *condición* **Entonces**

↳ Bloque de instrucciones a realizar cuando *condición* es verdadera

Sino

↳ Bloque de instrucciones a realizar cuando *condición* es falsa

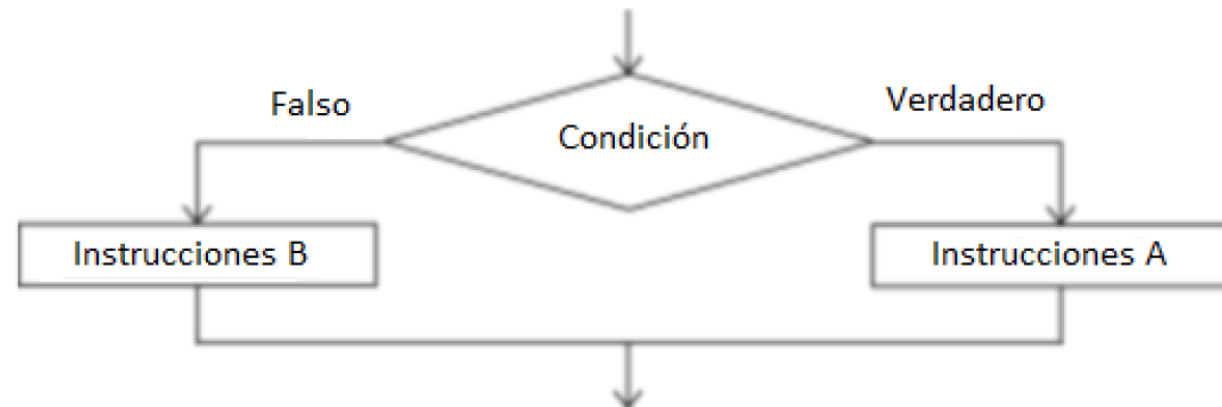
FinSi

SELECCIÓN DOBLE

El flujo de acciones es el siguiente:

- La primera parte de la selección doble continúa siendo igual a la simple, es decir, si la condición lógica es VERDADERO, se realizan las operaciones especificadas por el primer bloque de instrucciones.
- Pero si la condición es FALSO se realizan las instrucciones indicadas en el segundo bloque cuyo inicio se indica con las palabras **Sino**.

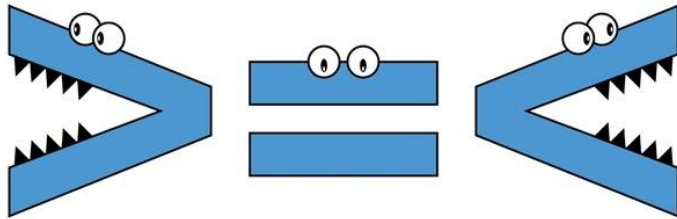
El fin de la selección es determinada por la instrucción **FinSi**.



Ejercicio: Mayor de dos números (usando selección doble)

PROBLEMA:

Suponga que se tienen dos números enteros y se desea compararlos para imprimir en pantalla cuál es el **mayor** de ellos.



En el cuaderno:



1. Identificar:

- Entrada: *Leer num1 y num2*
- Proceso: *Comparar:*
 - a) *Si $num1 > num2$*
 - b) *Si no, comparar:*
 - i. *Si $num2 > num1$*
 - ii. *Si no, entonces son iguales*
- Salida: *Mostrar mensaje indicando cuál número es mayor, o si son iguales.*

2. Dibujar diagrama de flujo

Ejercicio:

Contratación de un empleado (usando selección doble y AND)



PROBLEMA:

Suponga que se aplican dos exámenes a un aspirante al puesto. Sólo si obtiene calificación mayor a 80 en **ambos exámenes**, se le contratará. Mostrar en pantalla un mensaje que indique si será o no contratado.

En el cuaderno:

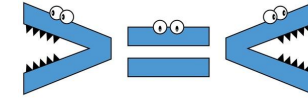


1. Identificar:

- Entrada:
- Proceso:
- Salida:

2. Dibujar diagrama de flujo

Práctica 11: Mayor de dos números (usando selección doble)



PROBLEMA:

Suponga que se tienen dos números enteros y se desea compararlos para imprimir en pantalla cuál es el **mayor** de ellos.

En el cuaderno ya hicimos:



- ✓ Entrada
- ✓ Proceso
- ✓ Salida

- ✓ Diagrama de flujo

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

~~Práctica 11:~~ **~~Contratación de un empleado~~** **~~(usando sólo selección simple)~~**



PROBLEMA:

Suponga que se aplican dos exámenes a un aspirante al puesto. Sólo si obtiene calificación mayor a 80 en ambos exámenes, se le contratará. Mostrar en pantalla un mensaje que indique si será o no contratado.

En el cuaderno:



- 1. Identificar:**
 - a) Entrada
 - b) Proceso
 - c) Salida

En PSeInt:



- 1. Escribir pseudocódigo.**
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
- 2. Generar diagrama de flujo.**
- 3. Guardar sus archivos.**

Práctica 12a:

Contratación de un empleado (usando selección doble y AND)



PROBLEMA:

Suponga que se aplican dos exámenes a un aspirante al puesto. Sólo si obtiene calificación **mayor a 80 en ambos exámenes**, se le contratará. Mostrar en pantalla un mensaje que indique si será o no contratado.



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 12b:

Contratación de un empleado (usando selección doble y OR)



PROBLEMA:

Suponga que se aplican dos exámenes a un aspirante al puesto. Sólo si obtiene calificación **mayor a 80 en al menos uno de los exámenes**, se le contratará. Mostrar en pantalla un mensaje que indique si será o no contratado.



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 12c:

Contratación de un empleado (usando selección doble)



PROBLEMA:

Suponga que se aplican dos exámenes a un aspirante al puesto. Si el aspirante tiene **mas de 80 en ambos exámenes**, se le contrata. Si obtiene **menos de 80 en uno** de los exámenes **pero mas de 90 en el otro**, pasa a una segunda ronda de selección. En otro caso, no se le contrata.

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 13a: Saludo personalizado por nombre (usando selección doble)

PROBLEMA:

Se le pregunta el nombre al usuario y se le manda un saludo personalizado con su nombre. Una respuesta vacía sería un caso erróneo, por lo tanto, sólo se enviará el saludo personalizado si la **longitud de la respuesta no es cero**. De lo contrario, se le manda un mensaje de error.

hola mundo!

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 13b: Saludo personalizado por nombre y edad (usando selección **doble y anidada**)

INSTRUCCIONES:

- Enviar un mensaje personalizado al usuario, dados su nombre y edad.
- Cuida que :
 - el nombre no sea vacío y
 - la edad esté en el rango [5,100].
- Enviar mensajes de error si no se cumple lo anterior, para cada caso.
- Si la edad es 18, mandaremos un mensaje felicitando al usuario por haber llegado a la mayoría de edad.
- **Hacerlo de tal manera que si no cumple con nombre válido, no puede pedir la edad. Y si no cumple con edad válida, no puede revisar si es mayor de edad.**

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 14: Calcular Índice de Masa Corporal (IMC) (usando selección **doble y anidada**) - GUIADA

INSTRUCCIONES:

- Validar de manera **anidada**:
 - Nombre no es vacío.
 - Edad en el rango [5,100] años.
 - Peso en el rango [30, 130] kgs.
 - Estatura en el rango [1.5, 2] mts.

- Calcular IMC $IMC = \frac{peso}{estatura^2}$
- Mostrar un mensaje relacionado a su peso:

IMC	Mensaje
<18.5	Estás por debajo del rango de peso normal para tu estatura
[18.5,25]	Estás en el rango de peso normal para tu estatura
>25	Estás por encima del rango de peso normal para tu estatura

En PSeInt:



- Escribir pseudocódigo.
 - Cabecera (Nombre, autor, descripción)
 - Comentarios por línea
- Generar diagrama de flujo.
- Guardar sus archivos.

Práctica 15a:

Día de la semana según el número (usando selección **simple**)



INSTRUCCIONES:

Dado un número entero entre 1 y 7 indique a que día de la semana corresponde.

En PSeInt:



1. **Escribir pseudocódigo.**
 - a) **Cabecera (Nombre, autor, descripción)**
 - b) **Comentarios por línea**
2. **Generar diagrama de flujo.**
3. **Guardar sus archivos.**

Práctica 15b:

Día de la semana según el número (usando selección **doble**)



INSTRUCCIONES:

Dado un número entero entre 1 y 7 indique a que día de la semana corresponde.

En PSeInt:



1. **Escribir pseudocódigo.**
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. **Generar diagrama de flujo.**
3. **Guardar sus archivos.**

¿Cuál es la diferencia entre ambas implementaciones?

Práctica 15a

**Selección
Simple**



Práctica 15b

**Selección
Doble**

Existe una estructura que simplifica mucho el uso de la selección simple y doble cuando se tienen casos como este.

SELECCIÓN MÚLTIPLE

Es una estructura de control que nos permite **simplificar un procedimiento** cuando se tienen que revisar **múltiples comparaciones** de equivalencia.

El formato es el siguiente:

Segun *variable* **Hacer**

número1: Bloque de instrucciones a realizar cuando *variable=número1*

número2: Bloque de instrucciones a realizar cuando *variable=número2*

...

De otro modo: Bloque de instrucciones a realizar cuando *variable*
toma un valor diferente a los enlistados

FinSegun

Donde *variable* puede ser de tipo entero únicamente (aunque versiones más nuevas de PSeInt ya aceptan cadenas de texto.)

Práctica 15c:

Día de la semana según el número (usando selección múltiple)



INSTRUCCIONES:

Dado un número entero entre 1 y 7 indique a que día de la semana corresponde.

En PSeInt:



1. **Escribir pseudocódigo.**
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. **Generar diagrama de flujo.**
3. **Guardar sus archivos.**

Práctica 16:

Indicar el mes según el número (usando selección múltiple)



INSTRUCCIONES:

Dado un número entero entre 1 y 12 indique a que mes del año corresponde.

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 17:

Menú de opciones (usando selección múltiple)



INSTRUCCIONES:

Presentar al usuario un listado de nombres de estados de la República Mexicana (5 para ejemplificar), codificados por un número entero, y pedirle que elija su estado de nacimiento utilizando el número que le corresponde en la lista.

Una vez que selecciona su estado de nacimiento, mandarle un mensaje indicando el gentilicio apropiado.

TAREA: Agregar el resto de los estados.

En PSeInt:



1. **Escribir pseudocódigo.**
 - a) **Cabecera (Nombre, autor, descripción)**
 - b) **Comentarios por línea**
2. **Generar diagrama de flujo.**
3. **Guardar sus archivos.**

Práctica 18:

Lanzar un par de dados y obtener la suma

GUIADA



INSTRUCCIONES:

Simular el lanzamiento de dos dados y sumar los resultados obtenidos.

Para simular el lanzamiento de un dado, se puede utilizar la función **Aleatorio(minimo,maximo)**, que regresa un valor entero en el rango [minimo,maximo].

Otra función para generar números aleatorios en PSeInt: **azar(limiteSup)**. Regresa un número aleatorio en el rango [0,limiteSuperior).

NOTA: No incluye el valor dado como límite superior.

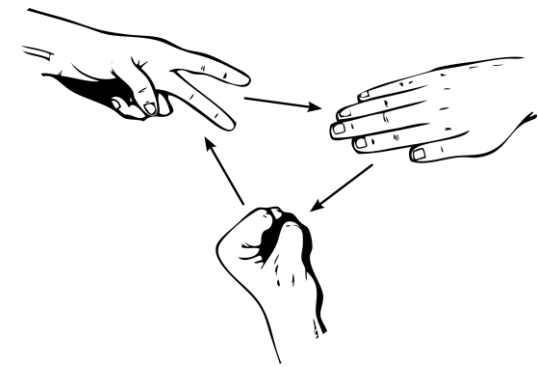
En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 19:

Juego: piedra, papel o tijera (usando selección múltiple)

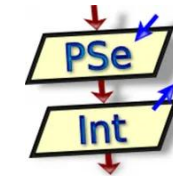


INSTRUCCIONES:

Para este juego se requieren dos jugadores. Cada uno de ellos debe seleccionar una de las opciones (piedra, papel o tijeras) de forma independiente.

Juegan		Regla	Ganador
Piedra	Papel	El papel cubre a la piedra	Papel
Piedra	Tijeras	La piedra rompe las tijeras	Piedra
Papel	Tijeras	Las tijeras rompen el papel	Tijeras

En PSeInt:



1. **Escribir pseudocódigo.**
 - a) **Cabecera (Nombre, autor, descripción)**
 - b) **Comentarios por línea**
2. **Generar diagrama de flujo.**
3. **Guardar sus archivos.**

Práctica 20:

Juego: el 7

(usando selección múltiple)



INSTRUCCIONES:

El jugador tira ambos dados y suma las caras superiores. Previamente, se hace una apuesta sobre lo que será la suma de los dados (abajo de 7, 7 ó arriba de 7).

En caso de que haya apostado a que la suma sería abajo del siete y acertó, gana la suma apostada. Lo mismo cuando apostó arriba del 7. En caso de que haya elegido el 7 y acierta, gana el triple de la apuesta. Obsérvese que no importan las combinaciones de números obtenidos, sino la suma de los mismos. En caso de no acertar, pierde la cantidad apostada. El usuario indica una cantidad inicial para empezar a jugar.

Es necesario considerar las variables que representen la *cantidad inicial*, la *apuesta*, la *selección del usuario*, los *dados* y la *suma* de los dados.

En PSeInt:



1. **Escribir pseudocódigo.**
 - a) **Cabecera (Nombre, autor, descripción)**
 - b) **Comentarios por línea**
2. **Generar diagrama de flujo.**
3. **Guardar sus archivos.**

**-- Hasta aquí Examen de
Primer Parcial --**





Estructuras de control

REPETICIÓN

REPETICIÓN (iterativas)

Comúnmente llamadas ciclos, bucles; permiten **ejecutar** un conjunto de instrucciones **varias veces** de forma automática.

Existen tres ciclos:

Mientras Hacer

Para Hacer

Repetir
Hasta Que

CICLO MIENTRAS HACER

Este ciclo necesita una **condición** (expresión lógica) para **controlar la repetición** del cuerpo del ciclo. La condición se evalúa primero para saber si el cuerpo del ciclo se va ejecutar o no.

El formato es el siguiente:

```
Mientras condición Hacer
// Bloque de instrucciones
// que se ejecutarán si
// condición es VERDADERO
FinMientras
```

Si la condición es verdadera, se ejecutan las instrucciones que conforman el cuerpo del ciclo hasta que se encuentra la señal de finalización: **FinMientras**. En ese momento, el control regresa al inicio del ciclo, es decir, a la evaluación de la condición. Cuando la condición es falsa, no se ejecuta el cuerpo del ciclo y se rompe.

CICLO MIENTRAS HACER: Ejemplos

Ejemplo 1: Cuando se sabe de antemano el **número de veces que se repetirá** la ejecución del ciclo. Para ello se utiliza una variable que **cuente** el número de veces que se ha ejecutado. A este tipo de variable se le conoce como **contador**. Cuando el contador llega a cierto valor, determinado de antemano, el ciclo se rompe:

```
Definir c Como Entero
c:=1
Mientras c<=5 Hacer
    Escribir "Hola, mundo."
    c:=c+1
FinMientras
```



```
Hola, mundo.
Hola, mundo.
Hola, mundo.
Hola, mundo.
Hola, mundo.
```

El contador (variable c) se inicializa en 1, y se incrementa de uno en uno con la operación $c:=c+1$ hasta alcanzar el valor de 6, donde la condición se evalúa en FALSO y el ciclo se rompe.

CICLO MIENTRAS HACER: Ejemplos

El control del ciclo está determinado por 3 operaciones sobre el contador:

Inicialización, Condición e Incremento,

los cuales pueden manipularse para que se repitan el mismo número de veces aunque los valores generados para el contador sean diferentes. Así, el segmento anterior puede cambiarse por el siguiente:

```
Definir c Como Entero
```

```
c:=15
```

```
Mientras c<=27 Hacer
```

```
    Escribir "Hola, mundo."
```

```
    c:=c+3
```

```
FinMientras
```



```
Hola, mundo.
```

```
Hola, mundo.
```

```
Hola, mundo.
```

```
Hola, mundo.
```

```
Hola, mundo.
```

En este caso, el contador se inicializa en 15, se incrementa de 3 en 3 con la operación $c:=c+3$ y se rompe el ciclo cuando sobrepasa el valor 27.

CICLO MIENTRAS HACER: Ejemplos

En general, un contador se incrementa con la operación

| contador := contador + constante

donde **constante** es un valor entero distinto de cero (negativo o positivo).

Así que es posible utilizar un contador que **decremente** su valor:

```
Definir c Como Entero  
c:=5  
Mientras c>0 Hacer  
    Escribir "Hola, mundo."  
    c:=c-1  
FinMientras
```



```
Hola, mundo.  
Hola, mundo.  
Hola, mundo.  
Hola, mundo.  
Hola, mundo.
```

CICLO MIENTRAS HACER: Ejemplos

Ejemplo 2: Un ciclo también puede usarse para **cuidar el rango de valores válidos** para una variable.

En el caso de que necesitemos leer el nombre del usuario y queremos asegurarnos de que no quede en blanco, se puede usar el siguiente segmento de código:

```
Definir nombre como cadena
Escribir "Cómo te llamas?"
Leer nombre
Mientras Longitud(nombre)=0 Hacer
    Escribir "ERROR: El nombre del usuario no puede quedar en blanco"
    Escribir "Cómo te llamas? " Sin Saltar
    Leer nombre
FinMientras
// TO-DO: Implementar el programa cuando se cumple el criterio del nombre
```

CICLO MIENTRAS HACER: Ejemplos

Ejemplo 3: Otro ejemplo sería, **asegurar que el valor de una variable numérica se encuentre dentro de un rango definido**, por ejemplo la edad:

```
Definir edad como entero
Escribir "Cuántos años tienes? [0,100] " sin saltar
Leer edad
Mientras edad<0 o edad>100 Hacer
    Escribir "ERROR: Se espera un valor en [0,100]"
    Escribir "Cuántos años tienes? [0,100] " sin saltar
    Leer edad
FinMientras
// TO-DO: Implementar el programa cuando se cumple el criterio de la edad
```

CICLO MIENTRAS HACER: Ejemplos

Ejemplo 4: Este ciclo también se puede usar para **asegurar que se elige una opción válida en un menú.**

En el siguiente ejemplo, si el usuario elige una opción que no está en el listado, se le muestra un mensaje de error y se le da la *oportunidad de elegir de nuevo*, esto gracias al uso de un ciclo:

```
Definir opUsuario Como Entero
```

```
Escribir "Menú de opciones"
```

```
Escribir "1) Opción1"
```

```
Escribir "2) Opción2"
```

```
Escribir "3) Opción3"
```

```
Escribir "4) Opción4"
```

```
Escribir "Elige una opción [1,2,3 o 4] " Sin Saltar
```

```
Leer opUsuario
```

```
Mientras opUsuario<1 o opUsuario>4 Hacer
```

```
    Escribir "ERROR: se espera un valor en el rango [1,4]"
```

```
    Escribir "... presiona cualquier tecla para continuar"
```

```
    Esperar Tecla
```

```
    Limpiar Pantalla
```

```
    Escribir "Menú de opciones"
```

```
    Escribir "1) Opción1"
```

```
    Escribir "2) Opción2"
```

```
    Escribir "3) Opción3"
```

```
    Escribir "4) Opción4"
```

```
    Escribir "Elige una opción [1, 2, 3 o 4] " Sin Saltar
```

```
    Leer opUsuario
```

```
FinMientras
```

```
// TO-DO: Implementar lo que se va a hacer en cada una de las opciones del menu
```

CICLO MIENTRAS HACER: Ejemplos

Otra variante que simplifica un poco el código anterior se puede lograr si aseguramos que la variable del usuario se inicialice con un valor que haga que la condición que controla el ciclo sea Verdadera (la primera vez) y una vez dentro del ciclo se comprueba si es válida:

```
opUsuario:=10 // inicializado asi para asegurar que entre al ciclo
Esperar Tecla
Mientras opUsuario<1 o opUsuario>4 Hacer
    Escribir "Menú de opciones"
    Escribir "1) Opción1"
    Escribir "2) Opción2"
    Escribir "3) Opción3"
    Escribir "4) Opción4"
    Escribir "Elige una opción [1,2,3 o 4] " Sin Saltar
    Leer opUsuario
    si opUsuario<1 o opUsuario>4 Entonces
        Escribir "ERROR: se espera un valor en el rango [1,4]"
        Escribir "... presiona cualquier tecla para continuar"
        Esperar Tecla
        Limpiar Pantalla
    FinSi
FinMientras
// TO-DO: Implementar lo que se va a hacer en cada una de las opciones del menu.
```

CICLO MIENTRAS HACER: Ejemplos

Ejemplo 5: Este ciclo también puede usarse para hacer que un código se repita mientras el usuario así lo desee. El valor que se usa como controlador del ciclo se le llama **Valor centinela**.

En el siguiente ejemplo, el ciclo se repite mientras la respuesta del usuario sea 'S' o 'SI', por lo que esos valores son los centinelas del programa.

Definir resp Como Caracter

Escribir "Quieres jugar? (S/SI) " Sin Saltar

Leer resp

resp:=Mayusculas(resp)

Mientras resp='S' o resp='SI' Hacer

// TO-DO: Aquí va la implementación del juego

Escribir "... estoy jugando" // BORRAR esta linea

Escribir "Quieres jugar otra vez? (S/SI) " Sin Saltar

Leer resp

resp:=Mayusculas(resp)

FinMientras

CICLO MIENTRAS HACER: Ejemplos

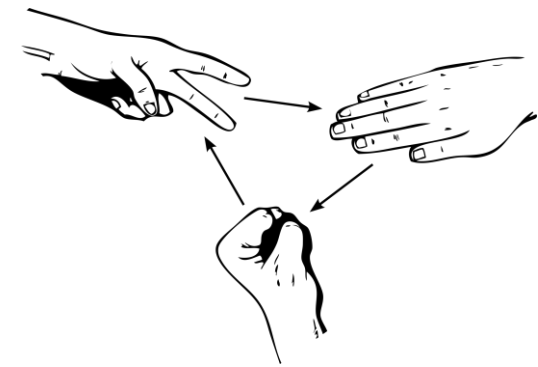
Ejemplo 6: Los contadores pueden servir también para contar la **ocurrencia** de algún suceso.

Por ejemplo, queremos contar cuántas veces aparece el número 0 en la captura del usuario: usamos un ciclo controlado por c1 donde, en cada iteración, se lee un valor y se revisa si es cero, en cuyo caso, se incrementa el valor de otro contador, c2. Al finalizar el ciclo, tendremos el c2 el número de 0's capturados por el usuario.

```
Definir c1,c2 Como Entero
Definir num Como Real
c1:=0
c2:=0
Mientras c1<=5 Hacer
    Escribir "proporciona un valor"
    Leer num
    Si num=0 Entonces
        c2:=c2+1
    FinSi
    c1:=c1+1
FinMientras
Escribir "Número de ceros capturados=",c2
```

Práctica 21:

Juego: piedra, papel o tijera (usando ciclo MIENTRAS HACER)



INSTRUCCIONES:

Reutiliza tu práctica anterior (P19), pero ahora usando un ciclo **Mientras Hacer** para controlar que el usuario elija un valor válido: Piedra, Papel o Tijeras.

El juego se repite mientras el usuario quiera seguir jugando.



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Guardar sus archivos.

Práctica 22:

Juego: Número mágico

(usando ciclo MIENTRAS HACER)



INSTRUCCIONES:

1. Se considera un valor oculto al usuario (llamado el número mágico) en un rango determinado [1,100]. El juego consiste en que el usuario tratará de adivinarlo.
2. Mientras el usuario no adivine el número mágico, se le dará otra oportunidad. Cada vez que el jugador falle, se le dará una pista sobre si el número mágico es mayor o menor al número dado.
3. Si el usuario falla 3 veces, pierde. Se le muestra mensaje mostrando el número mágico.
4. Si el usuario gana, se le muestra un mensaje felicitándolo y mostrando el número mágico.



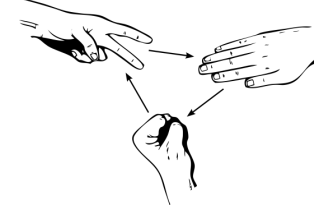
1. **Escribir pseudocódigo.**
 - a) **Cabecera (Nombre, autor, descripción)**
 - b) **Comentarios por línea**
2. **Guardar sus archivos.**

Práctica 23:

Sala de Juegos: El 7, PPT, Número mágico (usando ciclo MIENTRAS HACER)



7



INSTRUCCIONES:

Juntar los juegos que se han implementado hasta el momento en un solo programa:

1. Preguntar al usuario si quiere jugar.
2. Si la respuesta es afirmativa, presentar al usuario un menú con las opciones para jugar PPT, El 7 y el número mágico.
3. Después de la partida, preguntar al usuario si quiere jugar de nuevo. El procedimiento se repite mientras el usuario quiera seguir jugando.

En PSeInt:



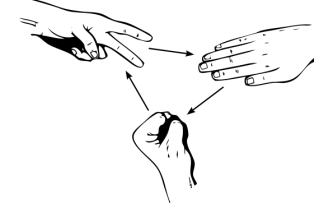
1. **Escribir pseudocódigo.**
 - a) **Cabecera (Nombre, autor, descripción)**
 - b) **Comentarios por línea**
2. **Generar diagrama de flujo.**
3. **Guardar sus archivos.**

Práctica 23b:

Sala de Juegos: El 7, PPT, Número mágico (usando ciclo MIENTRAS HACER)



7



INSTRUCCIONES:

Modifica la práctica anterior de tal modo que ahora el usuario pueda quedarse jugando más partidas del mismo juego, y cuando ya no quiera jugar, se regrese al menú principal para elegir otro juego o elegir salir de la sala de juegos (finalizar el programa).

En esta práctica, ejercitamos el anidamiento de ciclos Mientras-Hacer.

En PSeInt:



1. **Escribir pseudocódigo.**
 - a) **Cabecera (Nombre, autor, descripción)**
 - b) **Comentarios por línea**
2. **Generar diagrama de flujo.**
3. **Guardar sus archivos.**

CICLO MIENTRAS HACER: Ejemplos

Aparte de las variables contadores, otro tipo de variable muy utilizada en los ciclos son los **acumuladores**. Estos almacenan los valores acumulados de un conjunto de datos, comúnmente generados durante las iteraciones del ciclo.

En general, la operación de acumulación tiene el siguiente formato:

```
| acumulador = acumulador + variable
```

Donde *variable* es el valor acumulado, el cual puede ser positivo o negativo.

CICLO MIENTRAS HACER: Ejemplos

Ejemplo 7:

```
Definir c Como Entero
Definir suma,num Como Real
c:=0
suma:=0
Mientras c<5 Hacer
    Escribir Sin Saltar "proporciona un valor"
    Leer num
    suma := suma + num ← acumulador
    c := c + 1 ← contador
FinMientras
Escribir "La suma es ",suma
```

Práctica 24:

Promedio de un grupo de 5 alumnos

(usando ciclo MIENTRAS HACER)



INSTRUCCIONES:

Se pide una calificación por alumno, para obtener el promedio es necesario sumarlal y dividirlas entre el número de alumnos.

Variables requeridas:

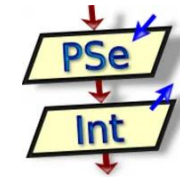
- **suma** para acumular las calificaciones
- **c** para contar el número de alumnos que se han considerado, y también para controlar el ciclo
- **calif** para leer la calificación del alumno en turno
- **prom** para el cálculo del promedio

Mientras la variable contadora tome un valor menor que 5, debe repetirse la ejecución de las instrucciones que forman parte del cuerpo del ciclo:

- Se solicita y se lee la calificación **calif**
- Se acumula en **suma**
- Se incrementa el contador **c**

Una vez terminado el proceso de repetición, se calcula el promedio y se muestra al usuario.

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.



Modifica para verificar que cada calificación esté en un rango válido de 0 a 100, antes de acumularla y contarla.

Práctica 25:

Encontrar el mayor de 8 números (usando ciclo MIENTRAS HACER)



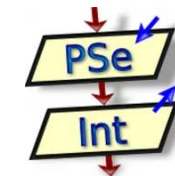
INSTRUCCIONES:

Se utiliza un contador *c*, que controle el número de iteraciones del ciclo, 8 en este caso, además se requiere una variable *num*, para guardar el número leído en cada vuelta. El mayor de los números encontrados en cada iteración se almacena en una variable auxiliar declarada para ello, *mayor*.

Nótese que es necesario leer el primer número *num* antes del ciclo para que éste sea considerado como el *mayor* hasta el momento. En el cuerpo del ciclo, cada vez que se lee un número, se compara con *mayor*, en caso de que éste sea mayor, se modifica el valor guardado en *mayor* sustituyéndolo con *num*.

Finalmente, el valor mayor encontrado es presentado como salida del programa.

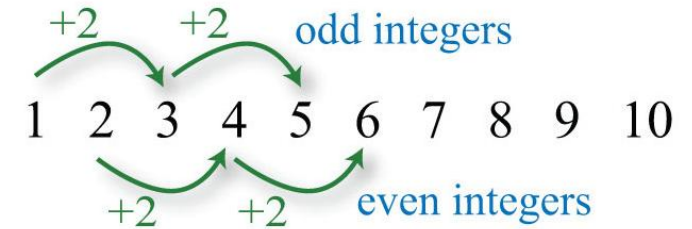
En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 26:

Sumar los números pares y los impares (usando ciclo MIENTRAS HACER)



INSTRUCCIONES:

Se suman los números pares e impares, por separado, que existen en el rango [1,100].

Son necesarias dos variables: *suma_pares* y *suma_impares*, para sumar los pares y los impares respectivamente. Se utiliza un contador *c*, para que tome todos los valores posibles en el rango dado. Es necesario revisar la paridad del contador para saber si su valor se acumula a la suma de los pares o de los impares.

Finalmente, cuando el ciclo se rompe, se imprimen los resultados obtenidos para ambas sumas.

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 27:

Factorial de n

(usando ciclo MIENTRAS HACER)



INSTRUCCIONES:

Calcula el factorial para un número n dado:

$$n! = \begin{cases} 1, & \text{si } n = 0 \\ 1 \cdot 2 \cdot 3 \cdot 4 \cdots n, & \text{si } n \geq 1 \end{cases}$$

Obsérvese que el factorial para un número negativo no está definido y deberá considerarse este hecho en el algoritmo. Se utiliza una variable **contador**, para controlar el ciclo y como auxiliar en el cálculo del factorial, f , ya que cada valor de contador será un término del producto acumulado en cada iteración del ciclo. Finalmente se muestra el resultado al usuario.

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

CICLO MIENTRAS HACER: Variante

Puesto que el cuerpo del ciclo Mientras-Hacer utiliza una **condición** para controlar la repetición, es posible utilizar una variable que tome valores **VERDADERO** o **FALSO** como controlador del flujo de acciones. Este tipo de variable se conoce como **lógica** o **booleana** y se declara de la siguiente manera:

| **Definir** identificador **Como** **Logico**

La forma de controlar el ciclo Mientras-Hacer mediante una variable lógica (comúnmente llamada **indicadora** o **bandera**) es asignar un valor inicial **VERDADERO** a dicha variable y hacer que la condición que controle el ciclo sea precisamente que la variable tome el valor **VERDADERO** y en algún momento de la ejecución del cuerpo del ciclo modificar el valor de la bandera a **FALSO** con lo cual se romperá el ciclo o viceversa (se inicializa como **FALSO**, la condición sería que la bandera tomara el valor **FALSO** y en algún momento se cambia a **VERDADERO** para que el ciclo se rompa).

CICLO MIENTRAS HACER: Variante

Ejemplo 8:

```
Definir dato Como Caracter
Definir ContinuarLectura Como Logico

ContinuarLectura := VERDADERO
Mientras ContinuarLectura Hacer
    Escribir "Teclea un caracter"
    Leer dato
    Si dato = '0' Entonces
        ContinuarLectura := FALSO
    FinSi
FinMientras
```

CICLO PARA HACER

Aunque el ciclo Mientras-Hacer nos ha sido útil cuando se conoce de antemano el número de veces que se ejecutará un bloque de instrucciones, el ciclo **Para-Hacer** es más adecuado para este caso.

Se utiliza una variable **contador**, para controlar el ciclo mediante las operaciones de *inicialización*, *condición* e *incremento*.

En la inicialización se asigna el **ValorInicial** al contador, el cual se incrementa hasta su **ValorFinal**, en cada iteración se incrementa en una cantidad especificada en **Incremento**.

El formato es el siguiente:

Para Contador **Desde** ValorInicial **Hasta** ValorFinal **Con Paso** Incremento **Hacer**

Bloque de instrucciones a realizar en cada iteración

FinPara

CICLO PARA HACER: Ejemplos

El formato de este ciclo permite algunas variantes, por ejemplo, si el incremento de la variable contador es de uno, puede omitirse, quedando el ciclo como sigue

```
Definir Contador Como Entero  
Para Contador Desde ValorInicial Hasta ValorFinal Hacer  
    //Bloque de instrucciones a realizar en cada iteracion  
FinPara
```

Otra variante permitida está relacionada con la forma en que se inicializa la variable Contador, permitiendo usar el operador de asignación, como se ilustra a continuación

```
Definir Contador Como Entero  
Para Contador<-ValorInicial Hasta ValorFinal Hacer  
    //Bloque de instrucciones a realizar en cada iteracion  
FinPara
```

Práctica 28:

Lanzamiento de una moneda 100 veces (usando ciclo PARA HACER)



INSTRUCCIONES:

Lanzar la moneda 100 veces y contar el número de veces que cayó cara y el número de veces que cayó cruz.

La simulación del lanzamiento de una moneda se realiza utilizando la función **aleatorio(0,1)**, en caso de que la función regrese un 1, se considera que se obtuvo una cara y se cuenta su ocurrencia utilizando el contador **NumCaras**; en caso contrario, se considera que cayó una cruz y se cuenta por separado, utilizando el contador **NumCruces**.

Se utiliza un ciclo **Para-Hacer** controlado por un contador **c**, el cual toma valores desde 1 hasta 100 con incrementos de 1 en 1; así, el ciclo se repetirá 100 veces. Finalmente, mostrar ambos contadores al usuario.



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

Práctica 29:

Factorial de n

(usando ciclo PARA HACER)



INSTRUCCIONES:

Calcula el factorial para un número n dado:

$$n! = \begin{cases} 1, & \text{si } n = 0 \\ 1 \cdot 2 \cdot 3 \cdot 4 \cdots n, & \text{si } n \geq 1 \end{cases}$$

Obsérvese que el factorial para un número negativo no está definido y deberá considerarse este hecho en el algoritmo. Se utiliza una variable **contador**, para controlar el ciclo y como auxiliar en el cálculo del factorial, f , ya que cada valor de contador será un término del producto acumulado en cada iteración del ciclo. Finalmente se muestra el resultado al usuario.

En PSeInt:



1. Escribir pseudocódigo.
 - a) Cabecera (Nombre, autor, descripción)
 - b) Comentarios por línea
2. Generar diagrama de flujo.
3. Guardar sus archivos.

CICLO REPETIR HASTA QUE

Este ciclo se utiliza cuando se sabe que el cuerpo del ciclo se va a **ejecutar al menos una vez**. La condición se evalúa al **finalizar** de ejecutarse las instrucciones que conforman el cuerpo del ciclo, si es verdadera, se repite la ejecución desde donde lo indica la palabra clave **Repetir**, en caso contrario, se rompe el ciclo.

Obsérvese que la única diferencia entre el ciclo Mientras-Hacer es el momento en que se realiza la evaluación de la condición.

El formato es:

Repetir

// Bloque de instrucciones a realizar en cada iteración

Hasta Que condición

CICLO REPETIR HASTA QUE: Ejemplos

Un uso común que se le da a este ciclo es para controlar que el usuario proporcione la información adecuada.

Así, por ejemplo, si queremos asegurarnos de que se proporcione una respuesta adecuada a una pregunta dada, utilizaremos este ciclo como "filtro" para que no se deje continuar el algoritmo hasta que se proporcione la respuesta correcta.

Ejemplo:

```
Proceso InicioJuego
  Definir resp Como Caracter

  Repetir
    Limpiar Pantalla
    Escribir "Quieres iniciar el juego? (s/n) "
    Leer resp

    Si resp<>'n' y resp<>'N' y resp<>'s' y resp<>'S' Entonces
      Escribir Sin Saltar "Se espera s/n "
      Esperar Tecla
    FinSi
  Hasta Que resp='n' o resp='N' o resp='s' o resp='S'
```

CICLO REPETIR HASTA QUE

Lenguajes que SI tienen este ciclo de manera nativa	• Pascal • Ruby • Perl • Otros...
Lenguajes que tienen un equivalente funcional (<i>do-while</i>)	• C • C++ • Java • C# • Javascript • Otros...
Lenguajes que NO tienen este ciclo	• Python

RESUMEN

- El ciclo **Mientras-Hacer** se utiliza cuando alguna operación a realizar repetidamente necesita supervisión previa ya que la revisión de la condición que lo controla se ejecuta antes que el cuerpo del ciclo. Es ideal utilizarlo si no se conoce el número de repeticiones necesarias para el proceso.
- El ciclo **Para-Hacer** se utiliza cuando tenga que repetirse la ejecución de un proceso un determinado número de veces. Este ciclo está controlado con una operación de inicialización, condición e incremento, aplicadas a un contador.
- Se utiliza el ciclo **Repetir-Hasta Que** cuando la ejecución de un proceso deberá realizarse por lo menos una vez ya que la revisión de la condición que lo controla se ejecuta después del cuerpo del ciclo.

RESUMEN

Ciclo	¿Cuándo se evalúa la condición?	¿Cuántas veces se ejecuta como mínimo?	¿Cuándo conviene usarlo?	Idea clave
Mientras-Hacer	Antes de ejecutar	0	Cuando no sabes cuántas veces se repetirá y puede ocurrir que no se ejecute ninguna vez.	“Mientras se cumpla la condición, repite.”
Para-Hacer	Se controla mediante un contador/rango	0 o el número determinado por el rango	Cuando conoces o puedes determinar de antemano cuántas veces quieres repetir algo.	“Repite N veces.”
Repetir-Hasta Que	Después de ejecutar	1	Cuando necesitas que las instrucciones se ejecuten al menos una vez y después decides si ya debes terminar.	“Hazlo una vez y después verifica si debe repetir.”

Bibliografía

- Rodríguez A. E. y Rodríguez C. I. Seudocódigo. Material Didáctico No. 5. Depto. de Matemáticas, Universidad de Sonora, 2002
- <https://pseint.sourceforge.net/index.php?page=portada.php>
- <https://pseint.org/>